



Programming in Haskell

UNIZG FER

September 30, 2025

Outline

- 1 Why Haskell?
- 2 Curriculum
- 3 Organization

Why Haskell?

- Haskell is the **flagship language** of functional programming

Why Haskell?

- Haskell is the **flagship language** of functional programming
- ... but why functional programming?

Why Haskell?

- Haskell is the **flagship language** of functional programming
- ... but why functional programming?
 - Employability

Why Haskell?

- Haskell is the **flagship language** of functional programming
- ... but why functional programming?
 - Employability
 - Functional concepts transfer to other languages

Why Haskell?

- Haskell is the **flagship language** of functional programming
- ... but why functional programming?
 - Employability
 - Functional concepts transfer to other languages
 - Fun and interesting

Why Haskell?

- Haskell is the **flagship language** of functional programming
- ... but why functional programming?
 - Employability
 - Functional concepts transfer to other languages
 - Fun and interesting
- Haskell is the **breeding ground** for bleeding-edge features and constructs

Why Haskell?

- Haskell is the **flagship language** of functional programming
- ... but why functional programming?
 - Employability
 - Functional concepts transfer to other languages
 - Fun and interesting
- Haskell is the **breeding ground** for bleeding-edge features and constructs
- Haskell is a **language designer's** favorite language

Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators Algebraic data types

Higher-kinded polymorphism Memory safety

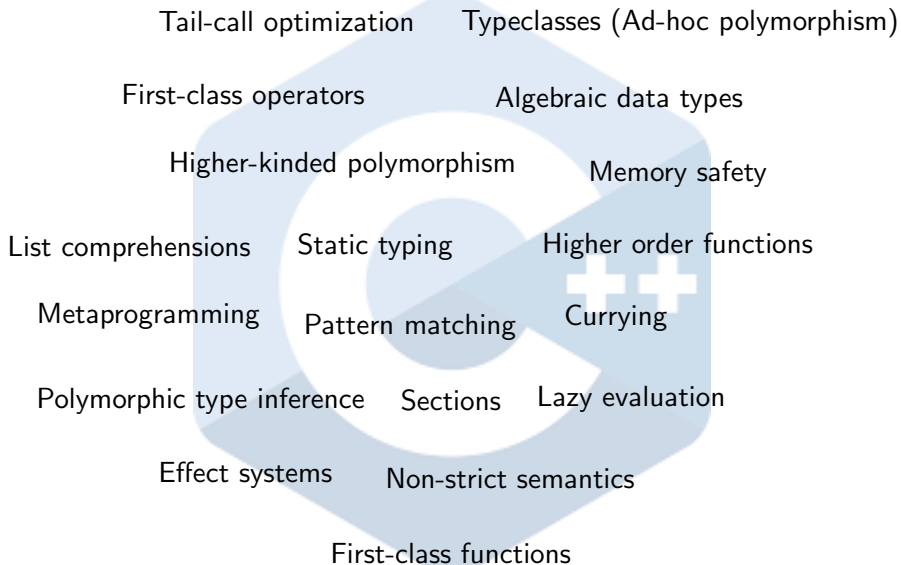
List comprehensions Static typing Higher order functions

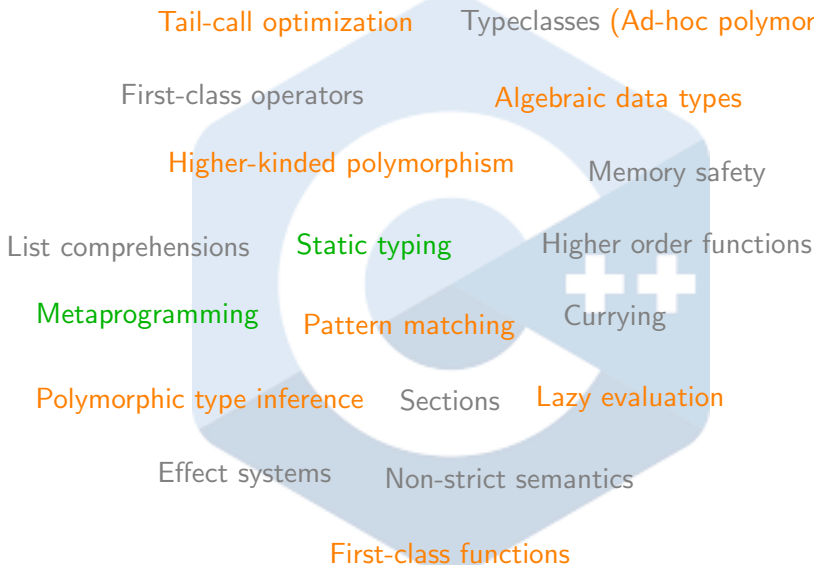
Metaprogramming Pattern matching Currying

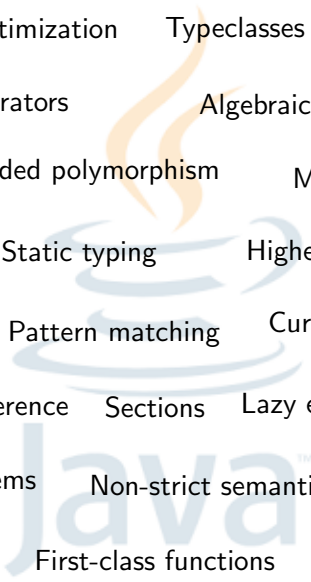
Polymorphic type inference Sections Lazy evaluation

Effect systems Non-strict semantics

First-class functions







Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators Algebraic data types

Higher-kinded polymorphism Memory safety

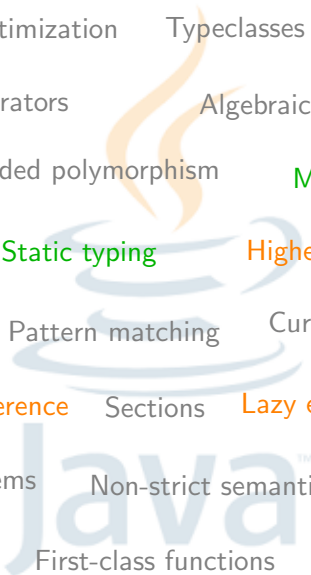
List comprehensions Static typing Higher order functions

Metaprogramming Pattern matching Currying

Polymorphic type inference Sections Lazy evaluation

Effect systems Non-strict semantics

First-class functions



Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators Algebraic data types

Higher-kinded polymorphism Memory safety

List comprehensions Static typing Higher order functions

Metaprogramming Pattern matching Currying

Polymorphic type inference Sections Lazy evaluation

Effect systems Non-strict semantics

First-class functions

Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators

Algebraic data types

Higher-kinded polymorphism

Memory safety

List comprehensions

Static typing

Higher order functions

Metaprogramming

Pattern matching

Currying

Polymorphic type inference

Sections

Lazy evaluation

Effect systems

Non-strict semantics

First-class functions

Tail-call optimization

Typeclasses (Ad-hoc polymorphism)

First-class operators

Algebraic data types

Higher-kinded polymorphism

Memory safety

List comprehensions

Static typing

Higher order functions

Metaprogramming

Pattern matching

Currying

Polymorphic type inference

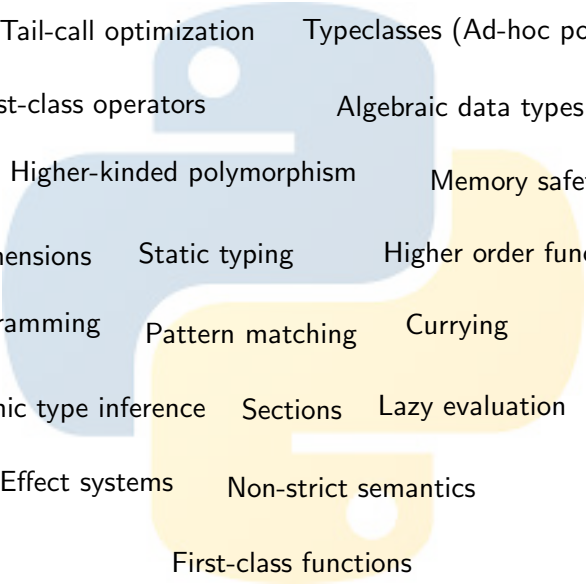
Sections

Lazy evaluation

Effect systems

Non-strict semantics

First-class functions



Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators Algebraic data types

Higher-kinded polymorphism Memory safety

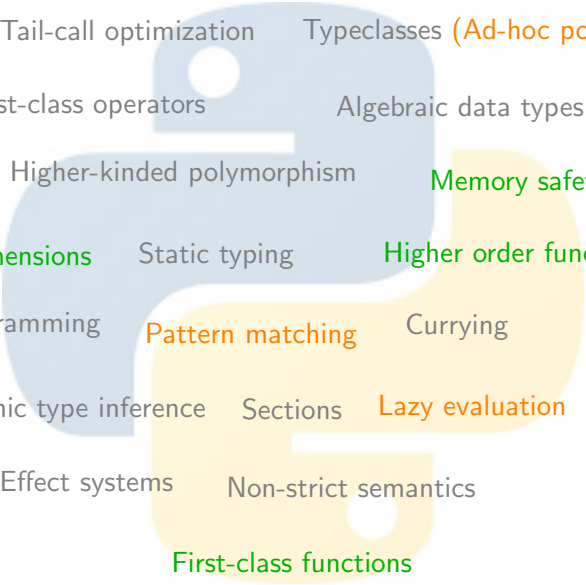
List comprehensions Static typing Higher order functions

Metaprogramming Pattern matching Currying

Polymorphic type inference Sections Lazy evaluation

Effect systems Non-strict semantics

First-class functions



Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators Algebraic data types

Higher-kinded polymorphism Memory safety

List comprehensions Static typing Higher order functions

Metaprogramming Pattern matching Currying

Polymorphic type inference Sections Lazy evaluation

Effect systems Non-strict semantics

First-class functions

Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators Algebraic data types

Higher-kinded polymorphism Memory safety

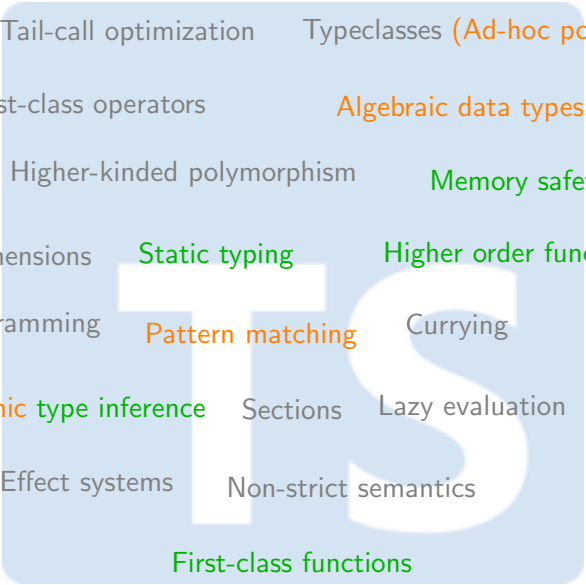
List comprehensions Static typing Higher order functions

Metaprogramming Pattern matching Currying

Polymorphic type inference Sections Lazy evaluation

Effect systems Non-strict semantics

First-class functions



Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators Algebraic data types

Higher-kinded polymorphism Memory safety

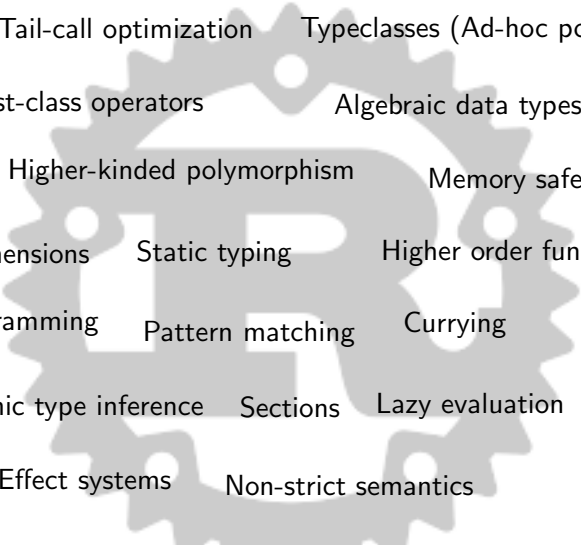
List comprehensions Static typing Higher order functions

Metaprogramming Pattern matching Currying

Polymorphic type inference Sections Lazy evaluation

Effect systems Non-strict semantics

First-class functions



Tail-call optimization Typeclasses (Ad-hoc polymorphism)

First-class operators Algebraic data types

Higher-kinded polymorphism Memory safety

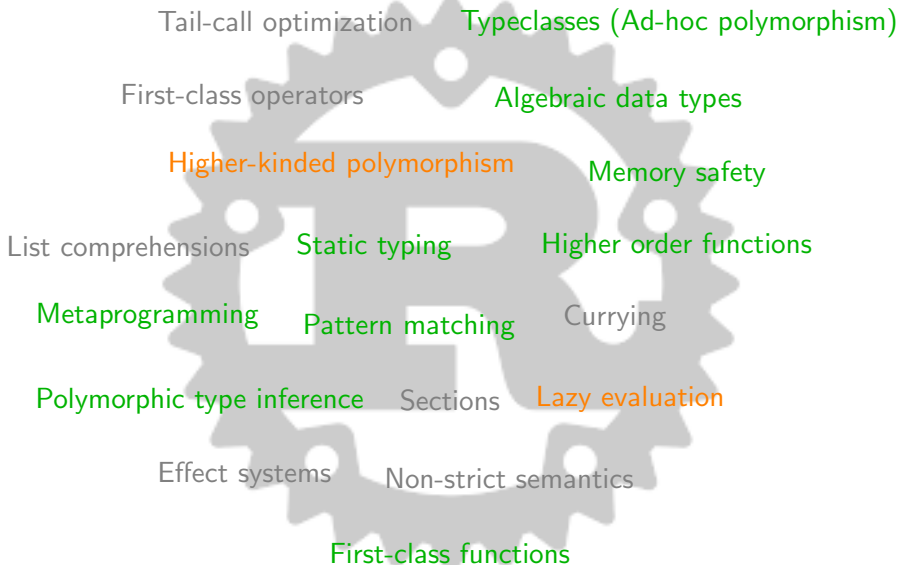
List comprehensions Static typing Higher order functions

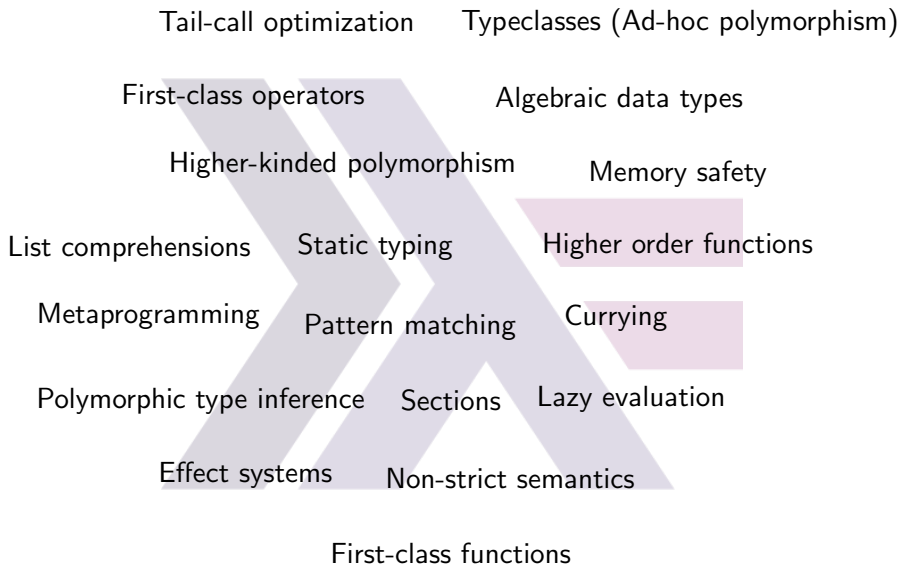
Metaprogramming Pattern matching Currying

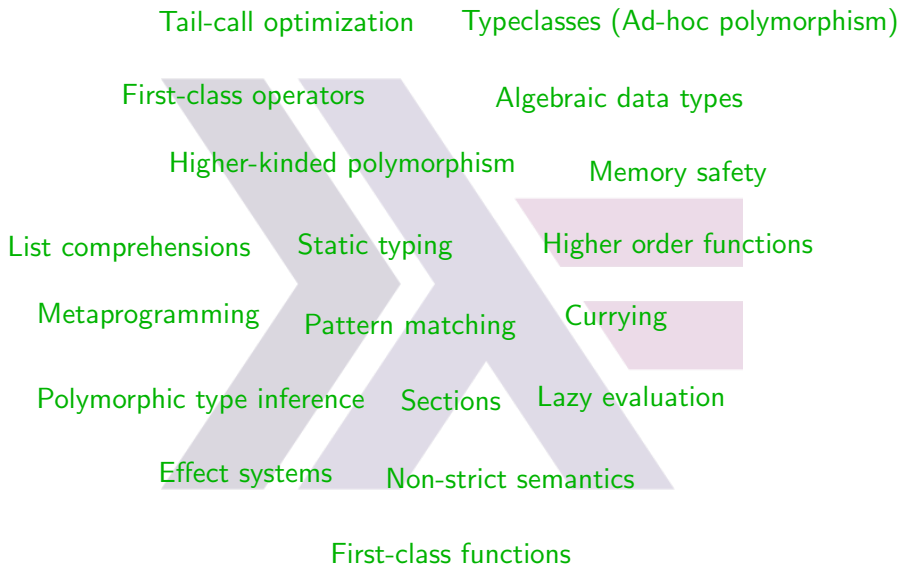
Polymorphic type inference Sections Lazy evaluation

Effect systems Non-strict semantics

First-class functions







- **Declarative** vs imperative

Haskell in a nutshell

- **Declarative** vs imperative
- **Statically-typed** vs dynamically-typed

Haskell in a nutshell

- **Declarative** vs imperative
- **Statically-typed** vs dynamically-typed
- **Functional** vs procedural vs object-oriented vs ...

Haskell in a nutshell

- **Declarative** vs imperative
- **Statically-typed** vs dynamically-typed
- **Functional** vs procedural vs object-oriented vs ...
- **Pure** vs allowing side effects

Haskell in a nutshell

- **Declarative** vs imperative
- **Statically-typed** vs dynamically-typed
- **Functional** vs procedural vs object-oriented vs ...
- **Pure** vs allowing side effects
- **Lazy** vs eager

Haskell in a nutshell

- **Declarative** vs imperative
- **Statically-typed** vs dynamically-typed
- **Functional** vs procedural vs object-oriented vs ...
- **Pure** vs allowing side effects
- **Lazy** vs eager
- **Type inference** vs manifest typing

- **Declarative** vs imperative
- **Statically-typed** vs dynamically-typed
- **Functional** vs procedural vs object-oriented vs ...
- **Pure** vs allowing side effects
- **Lazy** vs eager
- **Type inference** vs manifest typing
- **Nominal typing** vs structural typing

- **Declarative** vs imperative
- **Statically-typed** vs dynamically-typed
- **Functional** vs procedural vs object-oriented vs ...
- **Pure** vs allowing side effects
- **Lazy** vs eager
- **Type inference** vs manifest typing
- **Nominal typing** vs structural typing
- **Immutable** vs mutable

Outline

- 1 Why Haskell?
- 2 Curriculum
- 3 Organization

- Lists and tuples

- Lists and tuples
- Types and type classes

- Lists and tuples
- Types and type classes
- Syntax of functions

- Lists and tuples
- Types and type classes
- Syntax of functions
- Recursive functions

- Lists and tuples
- Types and type classes
- Syntax of functions
- Recursive functions
- Higher-order functions

- Lists and tuples
- Types and type classes
- Syntax of functions
- Recursive functions
- Higher-order functions
- Custom data types

- Lists and tuples
- Types and type classes
- Syntax of functions
- Recursive functions
- Higher-order functions
- Custom data types
- Custom type classes

- Lists and tuples
- Types and type classes
- Syntax of functions
- Recursive functions
- Higher-order functions
- Custom data types
- Custom type classes
- Input/output operations

- Lists and tuples
- Types and type classes
- Syntax of functions
- Recursive functions
- Higher-order functions
- Custom data types
- Custom type classes
- Input/output operations
- Monads

- Lists and tuples
- Types and type classes
- Syntax of functions
- Recursive functions
- Higher-order functions
- Custom data types
- Custom type classes
- Input/output operations
- Monads
- ... *If we have time:* **Industrial lecture** (Optional)

You will also ...

- Learn to use **Git** and **GitHub**

You will also ...

- Learn to use **Git** and **GitHub**
- Get professional **code reviews**

You will also ...

- Learn to use **Git** and **GitHub**
- Get professional **code reviews**
- Access a variety of learning resources (Haskell or otherwise)

You will also ...

- Learn to use **Git** and **GitHub**
- Get professional **code reviews**
- Access a variety of learning resources (Haskell or otherwise)
- Learn to use the **CLI** and other **industry-standard tools**

You will also ...

- Learn to use **Git** and **GitHub**
- Get professional **code reviews**
- Access a variety of learning resources (Haskell or otherwise)
- Learn to use the **CLI** and other **industry-standard tools**
- Learn more about programming languages in general

You will also ...

- Learn to use **Git** and **GitHub**
- Get professional **code reviews**
- Access a variety of learning resources (Haskell or otherwise)
- Learn to use the **CLI** and other **industry-standard tools**
- Learn more about programming languages in general
- Discuss anything you want with us (careers, Linux, editor setup, etc.)

Outline

- 1 Why Haskell?
- 2 Curriculum
- 3 Organization**

Lecturers



Filip Sodić



Mihovil Ilakovac



Luka Hadžiegrić



Franjo Mindek



Anton Vučinić



Frane Livić



Nikola Kraljević

Teaching assistants



Donik Vršnak



Jakov Schramadei



Marin Lovrinović



Miho Hren

Course Structure

- Lectures
 - Held in person
 - **Mandatory**, 1 absence allowed
 - Full schedule available on Ferweb (mostly Thursdays)

- Lectures
 - Held in person
 - **Mandatory**, 1 absence allowed
 - Full schedule available on Ferweb (mostly Thursdays)
- Training Exercises
 - Homeworks given after each lecture (give or take)
 - Submitted through **GitHub**
 - All homeworks must pass unit tests and TA code review

- Lectures
 - Held in person
 - **Mandatory**, 1 absence allowed
 - Full schedule available on Ferweb (mostly Thursdays)
- Training Exercises
 - Homeworks given after each lecture (give or take)
 - Submitted through **GitHub**
 - All homeworks must pass unit tests and TA code review
- Seminar
 - A small complete haskell project
 - Get to know what it means to develop a haskell application

To pass this course you must ...

- Attend lectures
- Complete all homework assignments on time (aka. *Training exercises*)
- Turn in the seminar
- *No exams*